

用五個 AI 各司其職寫出一篇好文章

給一個主題（例如「AA 制」），這個系統會自動產出 N 個版本的長文章。每個版本有不同的角色視角、不同的智慧錨點、不同的敘事風格。背後不是一個 AI 在寫，是五個專業分工的 AI Agent 一起協作完成。長文寫完後還可以選用第六個 Agent 自動剪出社群短版貼文。

一句話：這個系統在做什麼

一個主題字串，例如：

AA

N 個版本的長文章（3000 – 5000 字），每版角色、智慧錨點、風格都不同。例如三版分別是：故事體 + 哲思體 + 典故體。

傳統做法是：給 AI 一個主題，叫它寫文章。問題是這樣寫出來的東西很平、千篇一律、缺少深度。

我們的做法是：把「寫一篇好文章」拆成編輯部裡會出現的五種角色，每個角色用一個 AI Agent 扮演，讓他們依序協作。

為什麼要五個 Agent，不是一個？

一個 AI 同時要當熱點觀察者、卡點思考者、角色塑造者、博覽群書者、總編輯統籌者，會發生什麼事？答案是：每件事都做到 70 分，沒有一件 90 分。

但如果讓五個 AI 各做一件事，每個 AI 只專注在自己的視角上，整體品質就會跳起來。這就像真實的編輯部：寫稿的人不負責找選題，找選題的人不負責採訪，採訪的人不負責編輯定稿。

核心隱喻：把你的腦袋拆成五個專家，他們在你腦袋裡開會。每個人發言時別人不打斷，最後總編輯整合所有人的意見，寫出最終版本。

五個 Agent 的分工

熱點 Agent

找入口：從主題拓展出讀者第一秒會停下來的關鍵詞與情感鉤子。未來會接 Thread / Dcard 爬蟲。

model: haiku

卡點 Agent

找問題：把表面議題挖到底層。讀者真正卡住的不是「該不該 AA」，而是「AA 在親密關係裡定義了什麼是『我們』」。

model: sonnet

角色 Agent

找人性：產出 5-7 個不同背景、立場差異大的角色。每個角色有自己的歷史、口吻、內在矛盾，讓總編輯挑來寫不同版本。

model: sonnet

書籍 Agent

找智慧：從本地書庫（目前 9 本，未來 500+）找出能回應卡點的書與段落。這是整個系統最難的一塊，下面會詳細展開。

model: opus

總編輯 Agent

社群 Agent

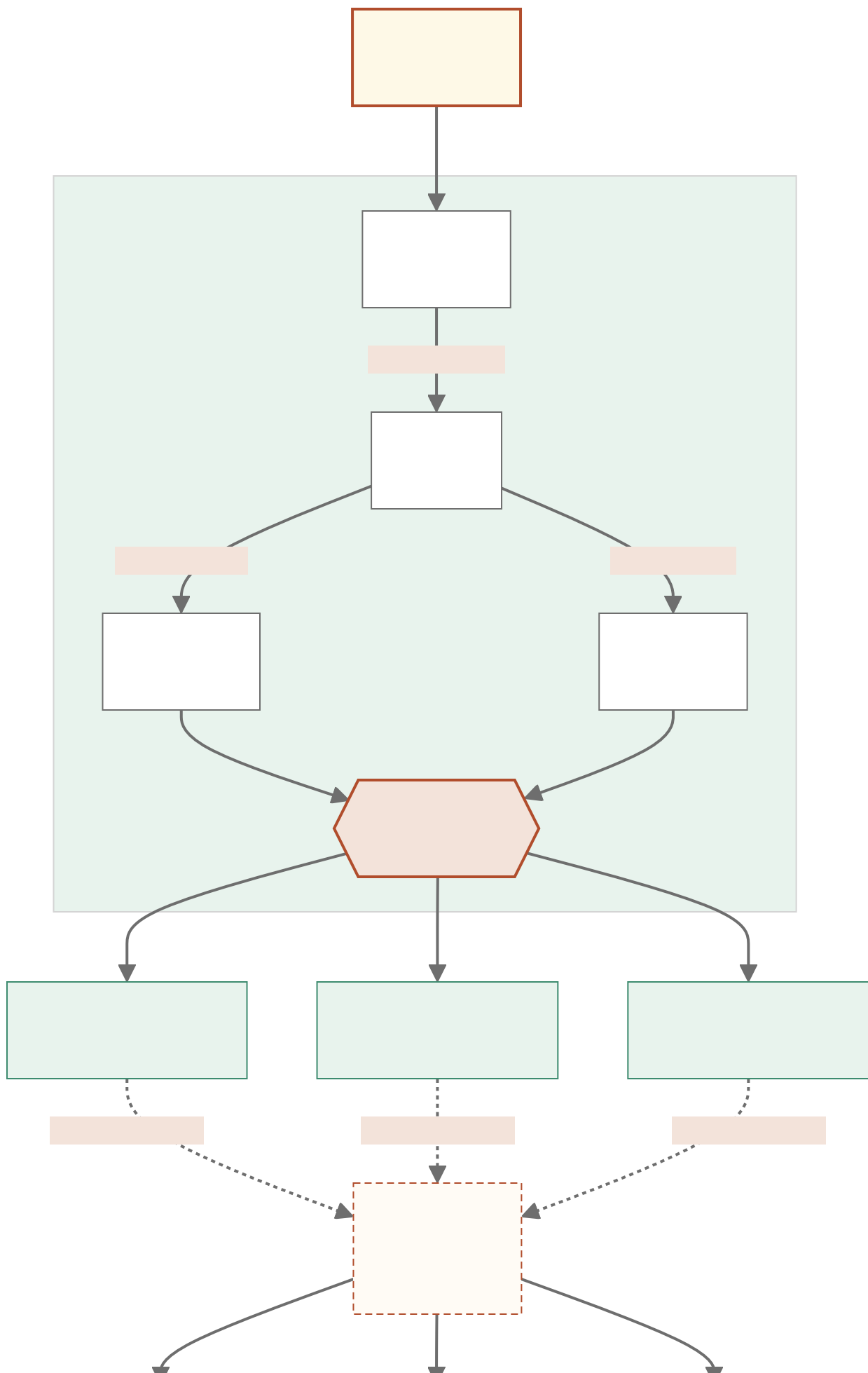
找靈魂：收齊前面四位的成果後，自己決定主軸、選 N 個對比強的角色、給每版指派一本書與一種敘事風格，然後寫稿。

model: opus

找鉤子：在每篇長文寫完後啟動，從原文剪出最有畫面/情緒/張力的故事片段當錨點，改寫成適合 Threads / IG / FB 的短版貼文。不是摘要，是把那顆鑽石單獨呈現。透過 `config.toml` 開關。

model: sonnet

整體流程





衍生：社群 Agent 怎麼把長文剪成貼文

長文寫好之後，常常還想配一個社群版（Threads / IG / FB）。但是直接讓 AI 「把這篇 4000 字濃縮成 300 字」，產出來幾乎都很無聊 — 因為它在做摘要，不是做鉤子。

核心觀念：社群貼文的目的是**鉤人**，不是傳遞資訊。所以對的做法是 — 在原文裡找出最有畫面、最有情緒、最有張力的那個 micro-story，剪下來、磨亮、單獨呈現。讀者沒看過原文也讀得懂，且讀完想去看完整版。

故事錨點要有四件事

人物

有名字、有清楚角色（太太、他媽媽）

情緒

愣住、心一沉、突然懂了

場景

時間、地點、感官細節

張力

A 以為 X，但 B 一句話戳穿

四種平台、四種口吻

threads	第一行直接進現場、一句一段、大量換行、口語	留白 / 問句 / 沒答案
ig	有氛圍感、段落稍長、可以有比喻	一句 takeaway，不雞湯
fb	段落正常、可以串多個 scene、最完整	可點題或問問題引留言
general	折衷	留白

關鍵設計：書名整個拿掉

長文裡的書（《非暴力溝通》、樊登在解讀某書時說……）在社群版會被整個移除，那句智慧轉成第一人稱重述：

「樊登在解讀《非暴力溝通》時說：『我們的感受根源於我們自身的需要。』」

「我朋友跟我講過一句話，我記到現在——你會生氣，從來不是因為對方做了什麼，是你某個需要沒被看見。」

原因：社群是「朋友式對話」的場域，引用書名會立刻出戲；硬塞書面引用節奏直接斷掉。

怎麼開啟

編輯 `config.toml`：

```
[social_post]
enabled = true
length = 300
platform = "threads" # ig / fb / general
```

跑完輸出資料夾會多出 `output_{i}_social.txt`，與長文 `output_{i}.txt` 並排：

```
output/v3/
├─ output_1.txt      ←
├─ output_1_social.txt ←
```

```
└─ output_2.txt
└─ output_2_social.txt
...
```

最難的問題：書籍 Agent 怎麼真的去「查書」

這是整個系統的核心難題，也是最值得花時間講清楚的部分。

問題長什麼樣

「書籍 Agent」要做的事聽起來很簡單：給定一個議題（例如「AA 在親密關係裡定義了什麼是『我們』」），從書庫裡找出能回應這個議題的書，並從書裡找出可以引用的具體段落。

但魔鬼在細節裡：

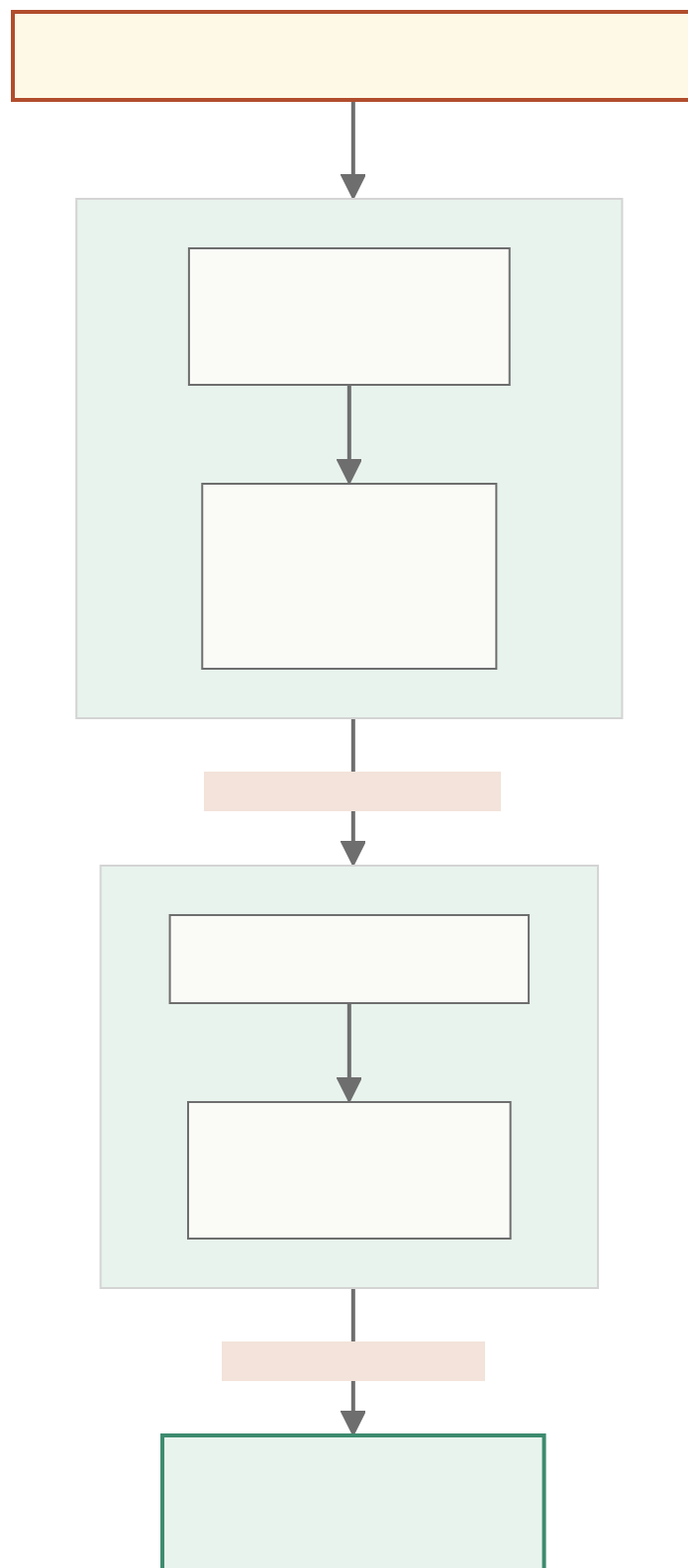
目前書庫有 9 本書、未來計劃擴到 500+ 本。
500 本逐字稿總共 ~29MB，相當於 700 萬個 token。任何 AI 模型的「記憶容量」都裝不下——即使是目前最大的 1M token 模型也不夠。

議題是「AA 制」，但相關的書裡寫的可能是「共同體」「互依」「界線」。這些字面上完全不像，但思想上是同一回事。傳統的關鍵字搜尋抓不到這種對應。

如果書籍 Agent 沒做好，最常見的失敗就是：**AI 開始瞎掰一本根本不在書庫裡的書**。它會講「《XXX》這本書裡提到…」，但那本書其實只存在於它的記憶裡，不在我們的本地書庫。這在內容生產上是致命的——讀者想去找原書會找不到。

解法：兩層檢索 (Two-Layer RAG)

我們的設計把「找書」拆成兩個截然不同的步驟，每一步用最適合它的技術：



為什麼一定要分兩層

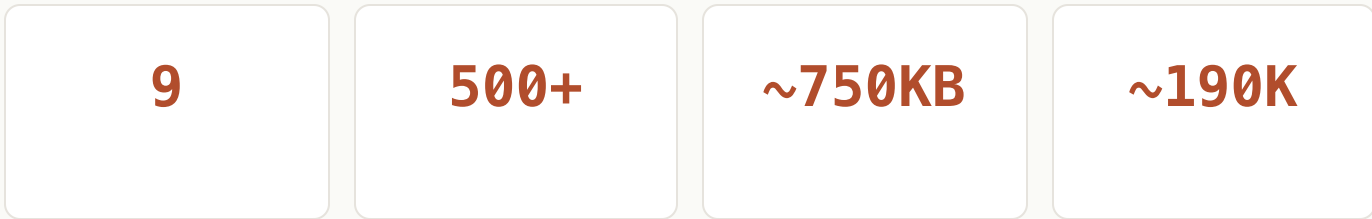
第一層 (概念)	「思想對應」 哪本書在談這個 議題的本質	LLM 直接讀全部書的 「目錄樹」 (精簡版 mindmap)	抽象概念匹配是 LLM 的強項。「AA」要連到 「共同體」需要推理，不是字面比對。
第二層 (段落)	「可引用原文」 具體哪段話最能 支撐論點	向量搜尋 (bge-m3 + chromadb)	具體文字匹配是向量的強項。資料量大 (500 本 ~6 萬段) 只能用向量過濾。

如果只用第一層 LLM 推理：模型很容易「想到對的方向，但編出不存在的原文」（也就是 hallucination）。

如果只用第二層向量搜尋：抓得到字面相關的段落，但抓不到抽象思想對應 — 因為「親密關係的共同體」跟「AA」字面差太遠。

兩層配合，才能既抓對方向，又拿到真實原文。

容量怎麼撐到 500 本？



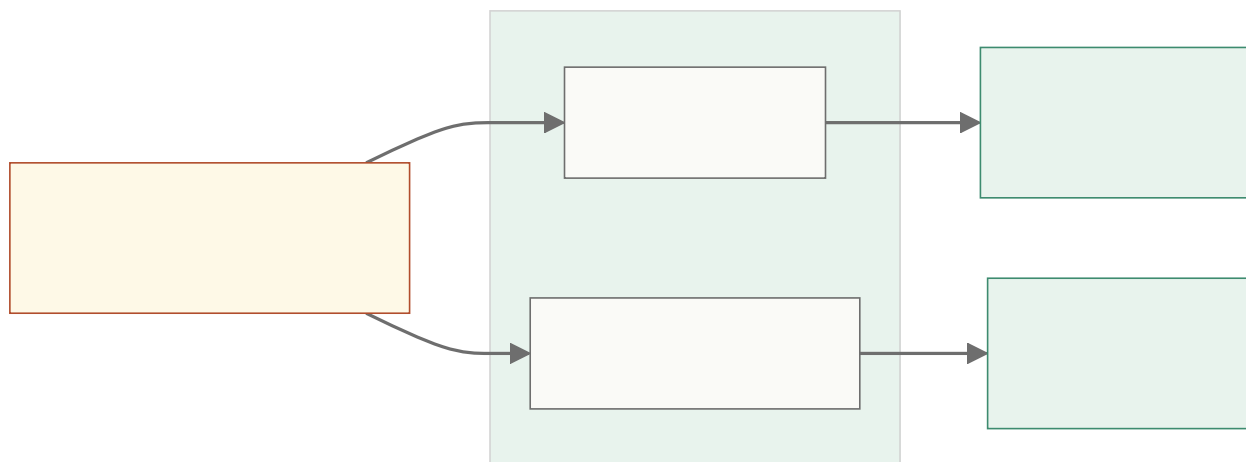
關鍵技巧：第一層只塞「精簡目錄樹」，不塞全文。每本書原本的逐字稿是 ~58KB，但目錄樹只有 ~1.5KB（標題 + 章節 + 重點概念）。500 本目錄樹加起來 750KB，相當於 19 萬 token，仍能完全塞進 1M context 的 Opus 模型一次讀完。

逐字稿則完全不進入 prompt — 它們躺在向量資料庫裡，第二層只取出 query 命中的段落（每次 ~15 段，~8K token）。

規模上限預估： ≤ 500 本可單次塞入；500 – 1000 本需要再加一層「類別過濾」（先選類別再選書）；> 1000 本需要更精緻的多階段檢索。當前設計把 hooks 留好（每本書的 metadata 已經有 tags 欄位），未來擴充不需要改 pipeline。

書庫怎麼準備（一次性 index 建置）

每本書放進系統時，會跑一次預處理腳本 `scripts/build_index.py`，做兩件事：



用的中文向量模型是 `BAAI/bge-m3`（在中文語意檢索上是目前最強的開源模型之一）。本地跑，零成本，一次 index 9 本書約 1 分鐘，500 本約 30 分鐘。

實際執行案例：以「AA 制」為例

把這個流程跑一次，五個 agent 之間實際傳遞了什麼：

熱點	「AA 制」	關鍵詞：婚後 AA、公平、家務分配；鉤子：「婚禮誓言裡沒有一句是各付各的」
卡點	熱點輸出	核心問題：「AA 在親密關係裡定義了什麼是『我們』？」 給書籍的搜尋詞：「親密關係 共同體 界線 互依」

角色	熱點+卡點	佳蓉（婚前 AA 擁護者，婚後懷疑）、雅婷（產後憂鬱、收入斷層）、家瑋（婚禮念過誓詞的丈夫）……共 5 – 7 個
書籍	卡點的搜尋詞	第一層讀目錄選 3 本：《非暴力溝通》《親密關係》《關鍵對話》 第二層找原文，每本回 1 段可引用內容
總編輯	以上全部	挑 3 個對比鮮明的角色 × 3 本書 × 3 種風格，寫出 3 個 4000 字長文

\$1.12

7.5 min

9 turns

3 calls

目前的限制與下一步

- 熱點 Agent 是 stub：目前只是把使用者輸入的主題往下傳。下一步接 Thread / Dcard 爬蟲，自動找熱貼、抓鉤子。
- 書籍 hallucination 偶發：即使 prompt 嚴格寫了「只能引用本地書庫」，模型偶爾還是會把它記憶中的書混進來。需要在最後加一個 programmatic 驗證層 — 自動掃描輸出檔，把任何不在書庫裡的書名標出來。
- 單台機器跑：四個 subagent 是循序執行的（後者依賴前者結果）。書籍跟角色其實可以併行，未來會優化。
- 沒有自動評估：目前靠人工比對 example 樣本判斷品質。未來可以加一個「品質 agent」自動打分（開頭抓人度、卡點深度、引用真實性、字數合理性）。

系統元件總覽

LLM 與 Embedding 的組合

第一層用 LLM 而非向量	抽象思想對應 embedding 抓不到，必須讓 LLM 推理
第二層用向量而非 LLM	具體文字匹配 + 大量資料 only embedding 撐得住
本地 bge-m3 而非雲端 embedding	中文 SOTA、零成本、500 本一次性 index 30 分鐘
Opus 給總編輯與書籍，Sonnet 給其他	總編輯與抽象書籍選擇需要強推理；其他工作 Sonnet 夠用，省成本
社群 Agent 寫一篇派一篇，不批次	每寫完一版立刻派發剪短，避免上下文遺失。批次處理會讓社群 agent 看不到原文細節
社群版主動移除書名	社群是「朋友式對話」場域，書面引用會立刻出戲、節奏斷掉

/Users/josephsyu/Not_In_Icloud/Vicky_Assistant/book_article_gen/